Light

GCSE Computer Science - Searching and Sorting Algorithms (Set 3)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • OCR J277 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Algorithms Searching Sorting

Multiple-choice topic test for **GCSE Computer Science (OCR J277)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

What change is needed to make bubble sort arrange items in descending order (largest to smallest) instead of ascending order?

- ○ **A.** Reverse the comparison operator - swap when the left item is smaller than the right, instead of when it is larger
- ○ **B.** Run the algorithm backwards through the list
- ○ **C.** Sort ascending first, then reverse the entire list
- ○ **D.** No change is needed - bubble sort always produces descending order

Q2 (1 mark)

Why would binary search be inappropriate for searching a list of 5 unsorted items?

- ○ **A.** Binary search only works on lists with more than 100 items
- ○ **B.** Binary search requires sorted data, and for only 5 items, sorting first then using binary search is more work than simply using linear search
- ○ **C.** Binary search cannot search for numbers, only text
- ○ **D.** 5 items is too few for any search algorithm to work

Q3 (1 mark)

If bubble sort makes no swaps during a complete pass through a list, what does this indicate?

- ○ **A.** The list is already sorted and no further passes are needed
- ○ **B.** The algorithm has encountered an error
- ○ **C.** The list contains only one item
- ○ **D.** The list needs to be reversed

Q4 (1 mark)

A list contains the values [4, 7, 2, 7, 9]. Linear search is used to find the value 7. Which position will be returned?

- ○ **A.** Both positions 1 and 3
- ○ **B.** Position 3 - the last occurrence of 7
- ○ **C.** Position 1 - the first occurrence of 7 is found and returned immediately

- ○ **D.** An error occurs because 7 appears twice

Q5 (1 mark)

What happens when a sorting algorithm is applied to an empty list (a list with zero items)?

- ○ **A.** The algorithm completes immediately - an empty list is already sorted by definition
- ○ **B.** The algorithm runs forever in an infinite loop
- ○ **C.** The program crashes with an error
- ○ **D.** The algorithm adds placeholder values to the list

Q6 (1 mark)

A sorted list is [2, 5, 5, 5, 8]. Binary search is used to find the value 5. Which position might be returned?

- ○ **A.** Always position 0
- ○ **B.** Always position 4
- ○ **C.** An error because there are multiple 5s
- ○ **D.** Position 2 (the middle 5), though the exact position depends on implementation details - binary search finds one occurrence but not necessarily the first or last

Q7 (1 mark)

What happens when insertion sort processes a list containing only one item?

- ○ **A.** The algorithm fails because there is nothing to compare
- ○ **B.** The algorithm completes immediately - a single item is already in sorted order
- ○ **C.** The item is deleted from the list
- ○ **D.** The single item is compared to itself

Q8 (1 mark)

In merge sort, why is a sub-list containing only one element considered sorted?

- ○ **A.** The algorithm automatically sorts single elements using a special method
- ○ **B.** Single elements are deleted before sorting begins
- ○ **C.** Single elements are always the number 1
- ○ **D.** A single element cannot be out of order relative to itself - there is nothing to compare it to, so it is sorted by definition

Q9 (1 mark)

After sorting the list [9, 3, 7, 1, 5] in ascending order, what position will the value 5 be at?

- ○ **A.** Position 0
- ○ **B.** Position 5
- ○ **C.** Position 4

○ **D.** Position 2 - in the sorted list [1, 3, 5, 7, 9], the value 5 is at the middle position

Q10 (1 mark)

Two different sorting algorithms are used on the same unsorted list. Assuming both work correctly, what can be said about their outputs?

○ **A.** The outputs will be identical - all correct sorting algorithms produce the same sorted result for the same input

○ **B.** The outputs will be completely different

○ **C.** Only one output will be correct

○ **D.** The outputs depend on the computer's speed

Q11 (1 mark)

What is the most efficient scenario scenario for linear search?

○ **A.** When the target item is at the very first position - only one comparison is needed

○ **B.** When the target item is in the middle of the list

○ **C.** When the target item is at the last position

○ **D.** When the target item is not in the list

Q12 (1 mark)

Why is data often sorted before being displayed to users?

- ○ **A.** Sorted data uses less storage space
- ○ **B.** Computers can only display sorted data
- ○ **C.** Sorted data is easier for humans to read, understand, and find specific items within
- ○ **D.** Unsorted data causes display errors

Q13 (2 marks)

Count the total number of swaps bubble sort makes to fully sort the list [3, 1, 2] into ascending order.

- ○ **A.** 0 swaps - the list is already sorted
- ○ **B.** 1 swap
- ○ **C.** 3 swaps
- ○ **D.** 2 swaps: first pass swaps 3 and 1, giving [1, 3, 2], then swaps 3 and 2, giving [1, 2, 3]. Second pass makes no swaps.

Q14 (2 marks)

Binary search is used on the sorted list [10, 20, 30, 40, 50, 60, 70] to find the value 20. Trace the search.

- ○ **A.** Check 40 (middle), 20 < 40 so search left half. Check 20 (middle of left half), found. Two comparisons total.
- ○ **B.** Check each value from 10 until 20 is found - 2 comparisons
- ○ **C.** Check 70, 60, 50, 40, 30, 20 - found after 6 comparisons

- ○ **D.** Check 20 directly - 1 comparison

Q15 (2 marks)

Insertion sort is used on [2, 1, 3]. How many comparisons are made in total?

- ○ **A.** 1 comparison
- ○ **B.** 2 comparisons: insert 1 (compare to 2, insert before), insert 3 (compare to 2, insert after)
- ○ **C.** 3 comparisons
- ○ **D.** 6 comparisons

Q16 (2 marks)

Two sorted sub-lists [1, 4] and [2, 3] are being merged. Show the order in which comparisons are made and elements are added to the result.

- ○ **A.** Add 1, 2, 3, 4 without any comparisons needed
- ○ **B.** Compare 1 and 2, take 1. Compare 4 and 2, take 2. Compare 4 and 3, take 3. Take 4. Three comparisons, result [1, 2, 3, 4].
- ○ **C.** Compare all elements to all other elements - 6 comparisons
- ○ **D.** Add 1, 4, 2, 3 - just concatenate the lists

Q17 (2 marks)

A school stores student names in alphabetical order. A teacher needs to check if 'Smith' is in the list of 200 students.
Which search method should be used and why?

- ○ **A.** Linear search because alphabetical order does not help searching
- ○ **B.** No search is possible on names, only numbers
- ○ **C.** Binary search because the data is already sorted alphabetically, allowing the search to eliminate half the remaining names with each comparison
- ○ **D.** Both methods would take exactly the same time

Q18 (2 marks)

Bubble sort (without early termination optimisation) is used on an already-sorted list of 10 items. How many passes will it make?

- ○ **A.** 0 passes - it detects the list is sorted and stops
- ○ **B.** 1 pass
- ○ **C.** 10 passes
- ○ **D.** 9 passes - unoptimised bubble sort always makes n-1 passes regardless of input order

Q19 (2 marks)

A teacher stores test scores in the order students submitted them: [78, 92, 65, 88, 71]. To find if any student scored exactly 88, which approach is correct?

- ○ **A.** Use binary search since it's faster
- ○ **B.** Sort the list first, then use binary search
- ○ **C.** It's impossible to search unsorted data

○ **D.** Use linear search directly on the unsorted list - it will check each score until finding 88 or reaching the end

Q20 (2 marks)

What does it mean for a sorting algorithm to sort 'in-place'?

○ **A.** The sorting happens instantly without any processing time

○ **B.** The algorithm rearranges items within the original list without needing significant extra memory for copies of the data

○ **C.** The items don't actually move - only their labels change

○ **D.** The sorting only works in one specific location in memory

Q21 (2 marks)

Insertion sort is used on a reverse-sorted list [5, 4, 3, 2, 1] to produce ascending order. Describe what happens.

○ **A.** The algorithm completes with no work needed

○ **B.** Only one swap is needed to fix the list

○ **C.** Each new element must be compared to all elements in the sorted portion and shifted to the very start, making this the least efficient scenario for insertion sort

○ **D.** Insertion sort cannot handle reverse-sorted data

Q22 (2 marks)

Binary search looks for value 1 in the sorted list [1, 3, 5, 7, 9]. Trace the search.

- ○ **A.** Check 5 (middle), 1 < 5 so search left. Check 1 (middle of [1, 3]), found. Two comparisons.
- ○ **B.** Check 5 (middle), 1 < 5 so search left [1, 3]. Check 3 (middle), 1 < 3 so search left [1]. Check 1, found. Three comparisons.
- ○ **C.** Check 1 directly at position 0 - one comparison
- ○ **D.** The value 1 cannot be found because it's at the edge

Q23 (2 marks)

Students are sorted by score, and two students (Amy and Ben) both have score 85. In the original list, Amy appears before Ben. After sorting with a stable algorithm, what is their order?

- ○ **A.** Ben will appear before Amy
- ○ **B.** Their order will be random
- ○ **C.** Amy will still appear before Ben - a stable sort preserves the original relative order of items with equal values
- ○ **D.** Both students will be merged into one entry

Q24 (3 marks)

A database of 50,000 product records is searched frequently throughout the day. Products are added monthly. Analyse whether to keep data sorted and use binary search, or keep data unsorted and use linear search.

- ○ **A.** Always use linear search because it's simpler to implement

○ **B.** Keep data unsorted to avoid sorting overhead

○ **C.** Keep data sorted and use binary search. Monthly additions can be inserted in sorted position with minimal disruption. Each binary search needs about 16 comparisons versus up to 50,000 for linear search. With frequent daily searches, the time saved vastly outweighs the small monthly sorting cost.

○ **D.** Neither approach works for databases this large

Q25 (3 marks)

Analyse bubble sort on the list [2, 4, 1, 3] step by step. Count total comparisons and swaps across all passes.

○ **A.** 3 comparisons, 3 swaps - sorting completes in one pass

○ **B.** Pass 1: 3 comparisons, 2 swaps, list becomes [2, 1, 3, 4]. Pass 2: 2 comparisons, 1 swap, list becomes [1, 2, 3, 4]. Pass 3: 1 comparison, 0 swaps, confirms sorted. Total: 6 comparisons, 3 swaps.

○ **C.** 12 comparisons, 6 swaps

○ **D.** The list cannot be sorted by bubble sort

Q26 (3 marks)

Analyse why merge sort uses the same amount of work regardless of whether the input is already sorted, randomly ordered, or reverse sorted.

○ **A.** Merge sort checks if the list is sorted before starting and skips work if so

○ **B.** Merge sort always divides the list to single elements regardless of order, then merges everything back. The divide phase does the same splits regardless of values. The merge phase does the same

comparisons because it cannot skip work based on initial order. This makes merge sort predictable but unable to exploit favourable inputs.

○ **C.** The amount of work varies greatly - sorted input is much faster

○ **D.** Merge sort uses different methods for different input arrangements

Q27 (3 marks)

A programmer writes a linear search that returns the wrong position. The code returns 'count' instead of 'count - 1' after finding a match. Analyse the effect.

○ **A.** The search will crash with an error

○ **B.** The search will return positions that are one higher than correct - if an item is at position 3, the function returns 4. This 'off-by-one' error means using the returned position would access the wrong item or potentially cause an out-of-bounds error for the last position.

○ **C.** The search will work correctly by coincidence

○ **D.** The search will find items faster due to skipping a decrement

Q28 (3 marks)

A small business needs to maintain a sorted list of 100 customer appointments. New appointments arrive one at a time throughout the day. Analyse why insertion sort is ideal for this scenario.

○ **A.** Insertion sort is the fastest sorting algorithm for all situations

○ **B.** The list is too small for any sorting to matter

○ **C.** Insertion sort excels at adding single items to already-sorted data. When a new appointment arrives, it only needs to find the correct position among the existing sorted appointments and insert

there. This requires checking at most 100 items and shifting some aside - much more efficient than re-sorting all 100 appointments with a different algorithm each time.

○ **D.** Binary search should be used instead of insertion sort

Q29 (3 marks)

A binary search implementation has a bug: when the target is not found, it enters an infinite loop instead of returning -1. Analyse what condition is likely missing or incorrect.

○ **A.** The algorithm is missing the comparison to check if target equals the middle element

○ **B.** The algorithm is missing or incorrectly implementing the check for when the search range becomes empty (when low > high). Without this termination condition, the algorithm keeps searching a range that cannot contain the target, never stopping.

○ **C.** The algorithm needs more memory to work correctly

○ **D.** Binary search always enters infinite loops when items are not found

Q30 (3 marks)

Compare how bubble sort, insertion sort, and merge sort would each perform on a list of 1,000 items that is already sorted. Analyse which algorithm is most efficient in this specific case.

○ **A.** All three algorithms perform identically on sorted input

○ **B.** Merge sort is fastest because it's generally the most efficient algorithm

○ **C.** Insertion sort is most efficient: it makes only 999 comparisons (each element compared once to confirm it's in place). Optimised bubble sort makes 999 comparisons in one pass detecting zero swaps.

Merge sort cannot detect sorted input and performs all its usual divide-and-merge work. For already-sorted data, insertion sort and optimised bubble sort are fastest.

○ **D.** Bubble sort is slowest regardless of input order

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.